

Configure IPv6 Addressing

Q1. Assign a static IPv6 address and verify it

Address used: **2001:db8:acad:1::10/64** (2001:db8::/32 is the range reserved for documentation and lab practice). Replace **Wi-Fi** or **eth0** with your own adapter name.

Windows (Command Prompt as Administrator)

```
ipconfig                                <- find the adapter name
netsh interface ipv6 add address "Wi-Fi" 2001:db8:acad:1::10/64
ipconfig                                <- verify
netsh interface ipv6 show addresses "Wi-Fi"    <- verify
```

Linux (Terminal)

```
ip link                                <- find the interface name
sudo ip -6 addr add 2001:db8:acad:1::10/64 dev eth0
ip -6 addr show dev eth0                <- verify
ifconfig eth0                           <- also works (net-tools)
```

Example output after configuration (your adapter details will differ)

```
Windows ipconfig:
    IPv6 Address. . . . . : 2001:db8:acad:1::10
    Link-local IPv6 Address . . . . . : fe80::a1b2:c3d4:e5f6:1234%12

Linux ip -6 addr show dev eth0:
    inet6 2001:db8:acad:1::10/64 scope global
    inet6 fe80::a00:27ff:fe4e:66a1/64 scope link
```

Verification: the address 2001:db8:acad:1::10 appears in the output. Optional: make it permanent on Linux with `nmcli con mod "Wired connection 1" ipv6.method manual ipv6.addresses 2001:db8:acad:1::10/64`. Commands run with `ip` are lost after a reboot. Take a screenshot of your own output for submission.

Q2. Unique local IPv6 address (fd...) and ping test

A unique local address (ULA) is in the range **fd00::/8** and works only inside a LAN, like private IPv4. Both devices must be on the same LAN and in the same /64.

Device	Unique local address
My PC	fd12:3456:789a:1::10/64
Other device on the LAN	fd12:3456:789a:1::11/64

Windows

```
netsh interface ipv6 add address "Wi-Fi" fd12:3456:789a:1::10/64
ping -6 fd12:3456:789a:1::11
```

Linux

```
sudo ip -6 addr add fd12:3456:789a:1::10/64 dev eth0
ping -6 fd12:3456:789a:1::11          (or: ping6 fd12:3456:789a:1::11)
```

On the other device, add fd12:3456:789a:1::11/64 with the same command. A successful ping shows replies like this:

```
Reply from fd12:3456:789a:1::11: time=2ms
Reply from fd12:3456:789a:1::11: time=1ms
Ping statistics: Sent = 4, Received = 4, Lost = 0 (0% loss)
```

If the ping fails: check that both devices are on the same network, and on Windows allow ICMPv6 echo requests: netsh advfirewall firewall add rule name="ICMPv6 Echo" protocol=icmpv6:128,any dir=in action=allow.

Q3. Temporary IPv6 address: does it change after reboot or reconnect?

The full steps and an observation sheet are in the text file **Task3_Temporary_IPv6.txt**. Summary of the method:

Windows

```
netsh interface ipv6 add address "Wi-Fi" 2001:db8:acad:1::50/64 ^
    validlifetime=300 preferredlifetime=150 store=active
ipconfig      <- then turn WiFi off/on and check again, then reboot and check again
```

Linux

```
sudo ip -6 addr add 2001:db8:acad:1::50/64 dev eth0 valid_lft 300 preferred_lft 150
ip -6 addr show dev eth0      <- lifetimes count down; then down/up the link, then reboot
```

Event	Normal behaviour of the temporary address
Time passes	Lifetime counts down. After the preferred time it is 'deprecated'; after the valid time it is removed.
Disconnect / reconnect	The manually added address is usually removed. An automatic privacy (temporary) address is replaced by a new random one.
Reboot	The address is gone because it was not saved (store=active, or the ip command). A saved static address would remain.

Q4. Zomato-style restaurant search using IPv6 addresses (Python)

The script stores restaurant names with their IPv6 addresses in a list, takes a partial name (not case sensitive) and prints every matching restaurant with its address. The same code is in **restaurant_search.py**.

```
# Zomato-style restaurant search using IPv6 addresses
import ipaddress

# Array (list) of restaurants: (name, IPv6 address)
restaurants = [
    ("Biryani Blues", "2001:db8:f00d::1"),
    ("Burger Barn", "2001:db8:f00d::2"),
    ("Pizza Palace", "2001:db8:f00d::3"),
    ("Punjabi Tadka", "2001:db8:f00d::4"),
    ("Dosa Delight", "2001:db8:f00d::5"),
    ("Chai Point Cafe", "2001:db8:f00d::6"),
    ("Pav Bhaji Corner", "2001:db8:f00d::7"),
    ("Thali House", "2001:db8:f00d::8"),
]

def search(partial):
    """Return all restaurants whose name contains the partial text."""
    partial = partial.strip().lower()
    return [(n, ip) for n, ip in restaurants if partial in n.lower()]

# Check that every stored address is a valid IPv6 address
for name, ip in restaurants:
    ipaddress.IPv6Address(ip)

print("Restaurant Search (type 'exit' to quit)")
while True:
    text = input("\nEnter part of a restaurant name: ")
    if text.strip().lower() == "exit":
        print("Goodbye!")
        break
    if not text.strip():
        print("Please type at least one letter.")
        continue
    matches = search(text)
    if matches:
        for name, ip in matches:
            print(f" {name} -> {ip}")
    else:
        print(" No restaurant found.")
```

Sample run

```
Restaurant Search (type 'exit' to quit)

Enter part of a restaurant name: pizza
Pizza Palace -> 2001:db8:f00d::3

Enter part of a restaurant name: bir
Biryani Blues -> 2001:db8:f00d::1

Enter part of a restaurant name: xyz
No restaurant found.

Enter part of a restaurant name: exit
Goodbye!
```

Run it with `python restaurant_search.py` (Windows) or `python3 restaurant_search.py` (Linux).

Q5. IPv6 subnetting scenario (AI-generated) and my configuration

Scenario: FoodieHub, a small food-delivery company, has the site prefix **2001:db8:acad::/48**. It must be split into four /64 subnets, one per department. The full scenario and steps are in **Task5_IPv6_Subnetting.txt**. The scenario was generated with an AI assistant (Claude).

Subnet	Department	Prefix	Gateway
1	Order Desk	2001:db8:acad:1::/64	2001:db8:acad:1::1
2	Kitchen	2001:db8:acad:2::/64	2001:db8:acad:2::1
3	Delivery Staff WiFi	2001:db8:acad:3::/64	2001:db8:acad:3::1
4	Servers	2001:db8:acad:4::/64	2001:db8:acad:4::1

My system is a PC in the Order Desk (Subnet 1), so I chose the valid address **2001:db8:acad:1::20/64** with gateway 2001:db8:acad:1::1. It lies inside the Subnet 1 prefix and is neither the network address nor the gateway.

Windows

```
netsh interface ipv6 add address "Ethernet" 2001:db8:acad:1::20/64
netsh interface ipv6 add route ::/0 "Ethernet" 2001:db8:acad:1::1
ipconfig
```

Linux

```
sudo ip -6 addr add 2001:db8:acad:1::20/64 dev eth0
sudo ip -6 route add default via 2001:db8:acad:1::1 dev eth0
ip -6 addr show dev eth0
```